

Задача "Проект каталог. Тесты представлений"

Вводная часть

Продолжаем работать над магазинчиком товара.

Не забываем общие требования заказчика к проекту:

- чтобы было красиво
- чтобы был поиск
- чтобы была фильтрация
- чтобы была возможность добавить, удалить и редактировать товар
- конечно, мы догадываемся, что завтра он может добавить новые пожелания...

Наша работа разбита на этапы. Сейчас реализуется **третий этап**.

Задание

Во этой части задания нужно:

1. Доработать каталог и добавить:

- Возможность вывода списка товара не только по алфавиту, но и упорядочив по другому атрибуту. В том числе по необязательному атрибуту.
- Возможность отобразить группу товаров по полю «один из».

2. Модифицировать структуру проекта:

- Объекты следует располагать в пакетах. То есть тесты должны быть в поддиректории tests, а не в одном файле, который создаёт Django по умолчанию (tests.py)
- В поддиректории созданы два файла с тестами:
 - test_routes.py - тесты маршрутов;
 - test_content.py - тесты проверки контента.

3. Разработать план тестирования и покрыть тестами модуль отображения каталога для вашего товара (сковороды, книги, электроника и т.д.).

Мы тестируем:

- ✓ Доступность маршрутов (URL → View)
- ✓ Статус-коды GET-запросов
- ✓ Содержимое страниц (контекст, шаблоны)

- ✓ Фильтрацию списка по полю типа
- ✓ Сортировку списка (сортировка по названию, по другому обязательному атрибуту, по необязательному атрибуту, по полю типа)
- ✓ Статические страницы (About)

Важно: На этом этапе мы **НЕ** тестируем:

- ✗ Авторизацию и права доступа
- ✗ Формы создания/редактирования
- ✗ POST-запросы на изменение данных

4. Документация: Тест-план

Создайте файл TEST_PLAN.md в корне приложения. Заполните 9 разделов:

№	Раздел	Требования к заполнению
1	Test Plan Identifier	Уникальный код (например, TP-PAN-2024-03)
2	Introduction	2-3 предложения: что тестируем, зачем, в рамках какой части курса
3	Test Items	Перечислите view-функции: ProductList, ProductDetail, AboutPage
4	Features to be tested	5-7 функций: список, фильтрация, детали, about, 404
5	Features NOT to be tested	Обязательно: авторизация, формы, POST-запросы, API
6	Approach	django.test.Client, GET-запросы, валидация контекста
7	Item Pass/Fail Criteria	Status 200/404, шаблон загружен, контекст содержит данные
8	Suspension Criteria	>30% тестов упали, ошибки БД
9	Test Deliverables	Код тестов, отчёт запуска, coverage

5. Матрица тест-кейсов

Минимум **10 тест-кейсов** для валидации сценариев:

ID	Сценарий	URL	Метод	Ожидаемый результат	Приоритет
TC-01	Главная страница каталога	/products/	GET	Status 200, шаблон list.html	High
TC-02	Список содержит объекты	/products/	GET	Status 200, context['products'] не пуст	High
TC-03	Фильтрация по типу	/products/?type=1	GET	Status 200, только товары типа 1	High
TC-04	Детальная страница	/products/<pk>/	GET	Status 200, context['product']	High
TC-05	Несуществующий товар	/products/999/	GET	Status 404	High
TC-06	Страница About	/about/	GET	Status 200, шаблон about.html	Medium
TC-07	About содержит текст	/about/	GET	Status 200, assertContains текст	Medium
TC-08	Пустой список товаров	/products/	GET	Status 200, список пуст (корректно)	Medium
TC-09	URL через reverse()	reverse('product-list')	—	Совпадает с /products/	Low
TC-10	Контекст имеет нужные ключи	/products/	GET	context содержит products, types	Medium

6. Отчёт о верификации

Приложите к сдаче:

Вывод запуска тестов

```
python manage.py test catalog.tests --verbosity=2
```

Покрытие кода

```
coverage run --source='.' manage.py test catalog.tests
```

```
coverage report
```

Примечания:

1. **Данные через админку.** В этой части курса данные создаются в `setUpTestData` или через админку Django. Формы создания — это следующий этап.
2. **GET-запросы только.** Не тестируйте POST в этой работе. Фокус на отображении, а не на изменении данных.
3. **Названия тестов.** Используйте шаблон: `test_<page>_<scenario>_<expected>`.
Пример: `test_detail_404_if_not_exists`.
4. **Контекст — это словарь.** `response.context` доступен как обычный dict. Проверяйте ключи и типы данных.
5. **Фильтрация через query params.** `/products/?type=1` — это GET-параметр, не путь. Используйте `request.GET.get('type')` в view.
6. **About — это тоже view.** Статическая страница должна иметь свой view, URL и тесты. Не забывайте про неё.
7. **Трассируемость.** Каждый тест должен покрывать конкретный пункт из матрицы тест-кейсов.